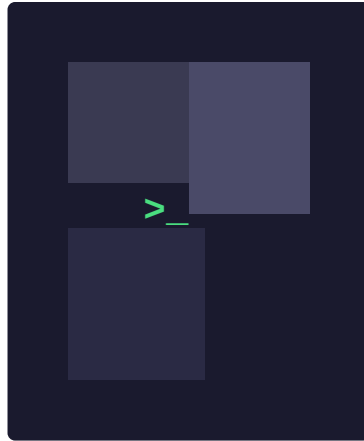




Claude Code — Module 00

CodeFlow

Summary:

This document contains the exercises for Module 00 of Claude Code.
Master your development agent: from /init to the SDK.

Version: 2.0

Contents

I	Introduction
II	General rules
III	Module-specific rules
IV	Exercise 00: Init & Context
V	Exercise 01: Session Management
VI	Exercise 02: HookGuard
VII	Exercise 03: PlayBot
VIII	Exercise 04: SDK Bridge
IX	Evaluation

Chapter I

Introduction

Claude Code is a command-line tool built by Anthropic that turns Claude into an autonomous development agent with direct access to your file system, terminal and workflow.

The goal of this module is for you to learn how to control, automate and extend Claude Code as a professional development tool. Using it is not enough: you must understand how its internal loop works, how to intercept it with hooks, how to manage context, and how to integrate it programmatically into your own projects.

Each exercise covers a different layer of control, from basic configuration to SDK integration from C++. By the end of the module you will be able to design workflows where Claude Code works for you in a supervised and efficient way.



Read the whole module before you start. Seriously, do it. Understanding the big picture will save you time on every exercise.

Chapter II

General rules

Environment requirements

- Claude Code installed and authenticated with a plan that allows Claude Code usage.
- Node.js ≥ 18 installed (required for SDK and MCP).
- jq installed on your system (sudo apt install jq or brew install jq).
- Git initialized in each exercise directory.
- Text editor of your choice (vim, nvim, VSCode, etc.).

Submission structure

Directories must be named: ex00, ex01, ex02, ex03, ex04. Each exercise includes a downloadable starter pack. Do not modify the starter's folder structure — add your files inside it.

Compilation (exercises with C++ code)

- Compile with c++ and the flags -Wall -Wextra -Werror.
- Your code must compile with -std=c++17 or higher.
- Each program must include a Makefile with: \$(NAME), all, clean, fclean, re.

Structure of each exercise

Each exercise has a mandatory part and an optional part (Bonus). The mandatory part must work completely in order to qualify for the bonus score. The bonus never makes up for an incomplete mandatory part.



Each exercise must be done on a separate git branch. Make a checkpoint commit BEFORE letting Claude Code modify your code. If you can't explain what each line does, don't submit it.

Chapter III

Module-specific rules

Claude Code is a powerful tool. With great power comes great responsibility (and a great API bill if you don't control your context).

- You must demonstrate that you understand each concept, not just that you've run it.
- In each exercise you must include a NOTES.md file with an explanation, in your own words, of what it does and why it works.
- Copying configurations from the internet without understanding them is not allowed. During evaluation you'll be asked why you made each decision.
- Claude Code must be used in interactive mode (not autonomous) unless the exercise explicitly says otherwise.
- Each exercise includes a starter pack. Download it before you start.



The goal is not for Claude Code to do your work. The goal is for YOU to control Claude Code. If during evaluation you can't explain your configuration, your grade will be 0.



It's recommended to read each exercise in full before starting. The exercises are progressive: each one builds on the previous one.

Chapter IV

Exercise 00: Init & Context

>_	Exercise: 00	Init & Context
	Directory: ex00/	
	Files: CLAUDE.md, NOTES.md, screenshots/	
	Forbidden: None	

Before asking Claude Code for anything, you need it to know where it is. This exercise covers project initialization and context control.

Starter pack included:

```
ex00/
├── Makefile
├── inc/
│   └── Calculator.hpp
├── src/
│   ├── main.cpp
│   └── Calculator.cpp
├── tests/
│   └── basic_test.cpp
└── NOTES.md
```

Mandatory part

1. Initialization

- Run `/init` inside the starter pack directory.
- Examine the generated `CLAUDE.md`. Document in `NOTES.md`: what Claude detected, what's missing, what you'd change.

2. Customizing `CLAUDE.md`

Modify `CLAUDE.md` to include at least:

- Compilation rules: flags, C++ standard, directory structure.
- Style restrictions: variable names, comment format.
- Directory limitation: Claude can only modify `src/` and `inc/`.
- Language instruction: responses in English.

3. References with @

- Use `@src/Calculator.cpp` so Claude only takes one file into account.
- Use `@tests/` to limit the context to the tests folder.
- Capture screenshots that prove Claude respects those limitations.

Expected output

- Customized CLAUDE.md with the 4 sections listed.
- NOTES.md with a critical analysis of the original vs customized CLAUDE.md.
- screenshots/ folder with at least 3 captures demonstrating the use of `@`.

Bonus

- Add a "common mistakes" section to CLAUDE.md with patterns Claude must avoid (e.g. don't use `printf`, don't modify the Makefile).
- Create a second CLAUDE.md in tests/ with testing-specific rules.



CLAUDE.md is the contract between you and Claude Code. If it's not well defined, Claude will do whatever it feels like. A good CLAUDE.md is the difference between a useful agent and a dangerous one.

Chapter V

Exercise 01: Session Management

>_	Exercise: 01	Session Management
	Directory: ex01/	
	Files: NOTES.md, session_log.txt, screenshots/	
	Forbidden: None	

Claude Code has a limited context window. If you don't manage it, it will start forgetting things, repeating itself, or consuming tokens unnecessarily. This exercise teaches you to control that window.

Starter pack included:

```
ex01/  
├─ session_scenario.md  
├─ session_log.template.txt  
└─ NOTES.md
```

Mandatory part

1. Guided scenario (mandatory for everyone)

Run the following complete scenario using the ex00 project:

- Step 1: Ask Claude to analyze the project structure.
- Step 2: Ask it to locate Calculator's main function.
- Step 3: Ask for a small improvement (add a new method).
- Step 4: Ask it to run make.
- Step 5: Introduce a deliberate error in the code.
- Step 6: Ask it to diagnose and fix it.
- Step 7: Run /compact.
- Step 8: Ask for a new modification related to step 3.
- Step 9: Observe and document what it remembers and what it needs to re-read.

2. Compact with instructions

- Run /compact [instructions] giving it specific priorities on what information to keep.
- Compare the result with the normal compact from step 7.

3. Clean & Escape

- Demonstrate the use of `/clean` to clear the history.
- Demonstrate the use of `Esc Esc` to interrupt an ongoing operation.
- Document in `NOTES.md`: when would you use `/compact` vs `/clean`? In what situation is the double escape critical?

Expected output

- `session_log.txt` with the full record of the scenario (9 steps).
- `NOTES.md` with before/after observations of `/compact`: what it remembered, what it lost, what it had to re-read.
- At least 2 of your own strategies to minimize token consumption.

Bonus

- Monitor token consumption during the session and generate a comparison table: operation → tokens consumed.
- Identify which kinds of operations consume the most context (reading large files, bash outputs, etc.).



Context is your most valuable resource in Claude Code. Managing it well is like managing memory in C: if you don't control it, it blows up in your face.

Chapter VI

Exercise 02: HookGuard

>_	Exercise: 02	HookGuard
	Directory: ex02/	
	Files: settings.json, hooks/, logs/, NOTES.md	
	Forbidden: None	

Hooks are the mechanism that turns Claude Code from an interactive tool into a supervised agent. In this exercise you'll implement hooks that protect and monitor Claude Code's work.

Starter pack included:

```
ex02/
├── settings.json
├── hooks/
│   ├── guard.sh
│   └── compiler_check.sh
├── logs/
│   └── .gitkeep
├── sample_hooks/
│   ├── bash_post.json
│   ├── write_post.json
│   └── read_post.json
└── NOTES.md
```

Mandatory part

1. Inspect with jq (ALWAYS first)

Before writing any functional hook:

- Create a PostToolUse hook with matcher "*" that dumps stdin to a log file (jq . > hook-log.json).
- Make Claude use at least 3 different tools (Read, Write, Bash).
- Examine the resulting JSON. Compare them with the starter's sample_hooks/. Document the structure of each one in NOTES.md.

2. PreToolUse — The guardian

Implement hooks/guard.sh that:

- Blocks any bash command containing `rm -rf` or `sudo`.
- Logs all attempted bash commands in `logs/commands.log` with a timestamp.

```
# Expected structure in settings.json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "bash hooks/guard.sh"
      }]
    }]
  }
}
```



In a real setup, absolute paths are recommended for hooks. This starter uses relative paths for didactic simplicity. As an advanced bonus, generate `settings.local.json` with absolute paths.

3. PostToolUse — The verifier

Implement `hooks/compiler_check.sh` that:

- After each Write on `.cpp` or `.hpp`, runs `make`.
- If it fails, the error is passed back to Claude so it can fix it.
- Logs each result (OK/FAIL + timestamp) in `logs/compile.log`.

Expected output

- `settings.json` with both hooks configured (Pre and Post).
- `hooks/guard.sh` working — blocks `rm -rf` and `sudo`.
- `hooks/compiler_check.sh` working — compiles after each edit.
- `logs/commands.log` with at least 5 timestamped entries.
- `logs/compile.log` with at least 3 entries (OK and FAIL).
- `NOTES.md` explaining the JSON structure of each tool.

Bonus

- Add a Notification hook that sends a message to a notifications.log file when Claude is idle for more than 60 seconds.
- Implement a Stop hook that runs make clean && make at the end of each Claude turn and logs the result.
- Generate a settings.local.json that uses absolute paths in all hooks. Document in NOTES.md why absolute paths are safer than relative ones in a production environment.



Always inspect with jq first (Part 1) before writing the functional hook. Don't assume the JSON structure — verify it. It's the equivalent of doing a debug printf before coding the logic.

Chapter VII

Exercise 03: PlayBot

>_	Exercise: 03	PlayBot
	Directory: ex03/	
	Files: mcp_config.json, NOTES.md, reports/, screenshots/	
	Forbidden: None	

MCP (Model Context Protocol) lets Claude Code connect with external tools. Playwright is a browser automation framework. Combined, Claude Code can browse the web, test interfaces and extract data from real pages.

Starter pack included:

```
ex03/
├─ mcp_config.json
├─ reports/
│   └─ .gitkeep
├─ screenshots/
│   └─ .gitkeep
└─ NOTES.md
```

Mandatory part

1. MCP configuration

- Install the Playwright MCP server.
- Configure Claude Code to detect it (mcp_config.json).
- Verify that Claude Code lists the available MCP tools.
- Capture a screenshot of the verification.

2. Data extraction

Ask Claude Code, using Playwright, to:

- Navigate to a public website (e.g. your GitHub profile).
- Extract specific information (repo names, bio, languages).
- Generate a report in reports/github_profile.md with the data.

3. Automated test

Design a test for Claude to run via Playwright:

- Verify that a page loads correctly (status 200).
- Check that a specific element exists on the page.
- Report the result (PASS/FAIL + timestamp) in reports/test_results.md.

Expected output

- mcp_config.json working with Playwright configured.
- reports/github_profile.md with real extracted data.
- reports/test_results.md with at least 2 tests (PASS/FAIL).
- screenshots/ with captures of the process.
- NOTES.md explaining the MCP pattern: configure, verify, use.

Bonus

- Configure a second MCP server (filesystem, GitHub, etc.) and show that Claude can use both in the same session.
- Combine a PostToolUse hook from ex02 with the Playwright MCP: after each web edit, log which page was visited.



Don't use Playwright to access private content, personal accounts with credentials, or for mass scraping. Public websites only.



MCP is Claude Code's plugin system. Playwright is just one example. The pattern you learn here (configure, verify, use) is the same for any MCP server.

Chapter VIII

Exercise 04: SDK Bridge

>_	Exercise: 04	SDK Bridge
	Directory: ex04/	
	Files: Makefile, main.cpp, CodeReviewer.{cpp,hpp}, bridge.mjs, NOTES.md	
	Forbidden: None	

The Agent SDK lets you call Claude Code programmatically from your own code. In this exercise you'll build a bridge between C++ and Claude Code: your C++ program will launch Claude Code via the SDK to review code automatically.

Starter pack included:

```
ex04/
├─ Makefile
├─ main.cpp
├─ CodeReviewer.hpp
├─ CodeReviewer.cpp
├─ bridge.mjs
├─ sample_report.json
├─ test_files/
│   └─ leak.cpp
│   └─ unittest.cpp
│   └─ logic_bug.cpp
├─ logs/
│   └─ .gitkeep
└─ NOTES.md
```

Mandatory part

This exercise is split into incremental steps. Complete each one before moving to the next.

Step 1: Read JSON from C++

- Read `sample_report.json` from your C++ program.
- Parse the JSON and print the issues to stdout in a readable way.
- You don't need the SDK yet — use the included example file.

```
// sample_report.json format
{
  "file": "leak.cpp",
  "issues": [
    {
      "line": 4,
      "type": "memory_leak",
      "message": "new without matching delete"
    }
  ],
  "summary": "1 issue found"
}
```

Step 2: SDK script

- Install: `npm install @anthropic-ai/claude-agent-sdk`.
- Complete `bridge.mjs` so it receives the path of a `.cpp` as an argument, sends it to Claude Code for analysis, and generates a JSON with the same format as `sample_report.json`.
- Use `allowedTools: ["Read"]` to limit what Claude can do.

Step 3: Connect C++ with the SDK

- Modify `CodeReviewer` to launch `bridge.mjs` with `system()`.
- Read the generated JSON and print it to `stdout`.
- Handle errors: file not found, SDK unavailable, malformed JSON.

Step 4: Validation with `test_files/`

- Run your program on the 3 files in `test_files/`.
- `leak.cpp` has a new without delete.
- `uninit.cpp` has an uninitialized variable.
- `logic_bug.cpp` has a logic error.
- Verify that Claude detects the issues in each one.

Expected output

- `CodeReviewer` compiled and working.
- `bridge.mjs` that generates valid JSON.
- Successful execution on the 3 `test_files`.
- `NOTES.md` explaining the full flow: `C++ → system() → bridge.mjs → SDK → Claude → JSON → C++ shows result`.

Bonus

- Integrate a Stop hook from ex02: when Claude finishes, automatically run CodeReviewer on the modified files and log the result in logs/review.log with a timestamp.
- Replace `system()` with `fork()/exec()` to launch `bridge.mjs`. `system()` is simpler but less safe; `fork()/exec()` is more robust and demonstrates systems-programming knowledge.

Final bonus: remote notification

Take the flow one step further: instead of logging the review result to a .log file, send it as a message to Telegram or Slack.

- Create a Telegram bot with @BotFather and get the token.
- Modify the Stop hook so that, after running CodeReviewer, it sends the summary via curl to the Telegram bot.
- The message must include: file name, number of issues, timestamp and a summary of the errors.
- Alternative: use a Slack webhook if you prefer.

```
# Example: send result to Telegram from the hook
TOKEN="your_bot_token"
CHAT_ID="your_chat_id"
MSG="Review: $FILE - $ISSUES issues found"
curl -s -X POST "https://api.telegram.org/bot$TOKEN/sendMessage" \
  -d chat_id="$CHAT_ID" -d text="$MSG"
```



This bonus closes the full loop of the module: Claude edits code, the hook launches the review via the SDK, and the result reaches your phone. You kick off the process, head to CrossFit, and the result arrives.

Chapter IX

Evaluation

Submit your work in your Git repository. Only the work inside your repository will be evaluated during the defense.

During evaluation you may be asked to:

- Explain every line of your hooks and configurations.
- Modify a hook live to add a new feature.
- Demonstrate that you understand the full flow: from `/init` to the SDK.
- Explain the decisions in your `CLAUDE.md` and why you chose those rules.
- Run your CodeReviewer on a new file the evaluator gives you.

The `NOTES.md` of each exercise is **MANDATORY**. Without it, the exercise is graded 0 regardless of whether it works.



The goal of this module is not for Claude Code to work for you. It's for you to demonstrate that you control Claude Code. If you can't explain your work, it's not your work.



By Toutatis! Use your brain! Claude Code is a tool, not a substitute for your reasoning ability.